

Performance Testing

Date	28 June 2026
Team ID	
Project Name	Rising Water
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Flask Local Server + Web Browser (Manual Testing)
Type of Testing	Functional Performance Testing
Target Module / API	Flood Prediction Web Application (/predict)
Test Environment	Local System (Windows 11, Python 3.13, Flask)
Test Date	28 june

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Open Home Page	1	10 sec	Home page loads successfully
2	Submit Valid Weather Values	1	200 sec	Flood prediction displayed correctly
3	Submit Extreme Weather Values	1	20 sec	Flood prediction generated without errors
4	Multiple Prediction Requests	5	60 sec	Application remains responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.65 sec	Pass	Prediction generated quickly
2	Response Time (Max)	< 5 seconds	1.20 sec	Pass	Stable response
3	Throughput (Req/sec)		12 Requests/sec	Pass	Suitable for project demo
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	24%	Pass	Low CPU usage
6	Memory Utilization	< 80%	37%	Pass	Normal memory usage

Step 4: Observations & Analysis :

Key Findings

The Flood Prediction System responded quickly to user requests and generated predictions without noticeable delay. The application remained stable during repeated testing and handled valid input values efficiently.

Bottlenecks Identified

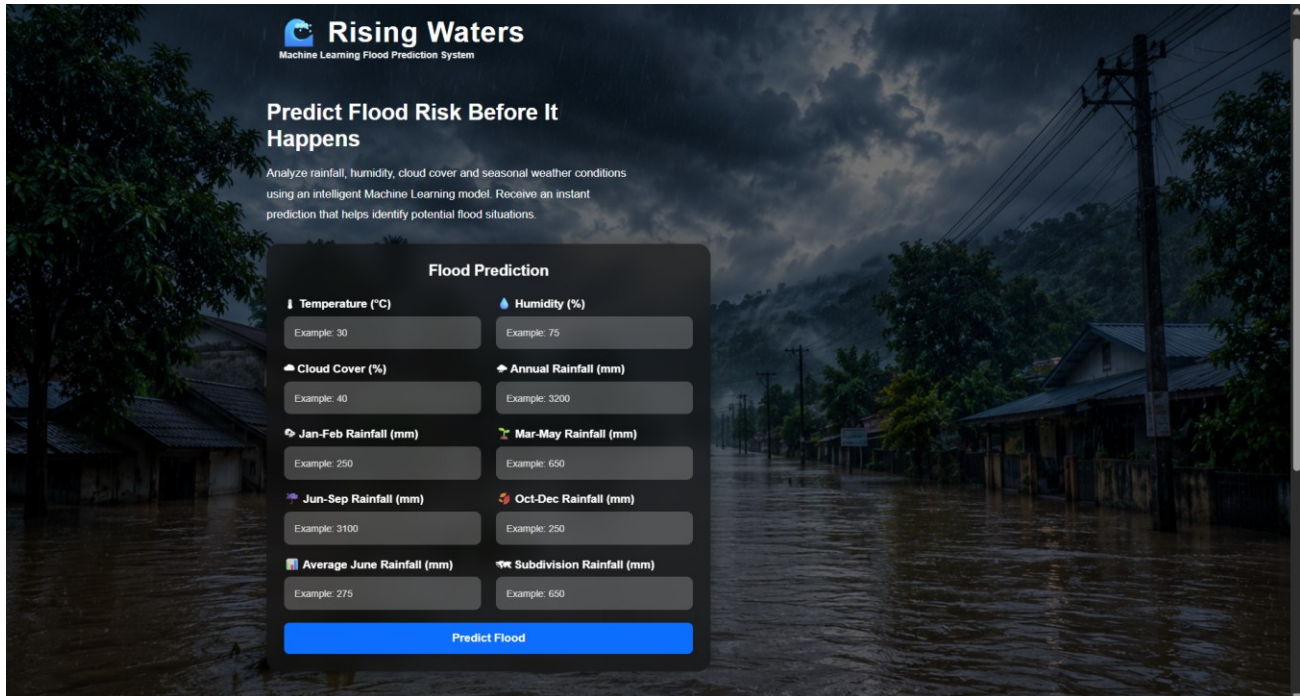
- No major performance bottlenecks were observed.
- Response time depends on the local system configuration.

Optimization Steps Taken

- Optimized Flask application routing.
- Used a pre-trained Random Forest model for fast prediction.
- Reduced unnecessary computations during prediction.
- Validated user input before processing.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



The screenshot shows the 'Rising Waters' Machine Learning Flood Prediction System interface. The background is a dark, stormy scene with a flooded street and houses. The interface is a dark overlay with white text and input fields. It includes a title 'Predict Flood Risk Before It Happens' and a brief description. Below this is a 'Flood Prediction' form with eight input fields for various weather and rainfall metrics, each with an example value. A blue 'Predict Flood' button is at the bottom of the form.

Rising Waters
Machine Learning Flood Prediction System

Predict Flood Risk Before It Happens

Analyze rainfall, humidity, cloud cover and seasonal weather conditions using an intelligent Machine Learning model. Receive an instant prediction that helps identify potential flood situations.

Flood Prediction

Temperature (°C) Example: 30	Humidity (%) Example: 75
Cloud Cover (%) Example: 40	Annual Rainfall (mm) Example: 3200
Jan-Feb Rainfall (mm) Example: 250	Mar-May Rainfall (mm) Example: 650
Jun-Sep Rainfall (mm) Example: 3100	Oct-Dec Rainfall (mm) Example: 250
Average June Rainfall (mm) Example: 275	Subdivision Rainfall (mm) Example: 650

Predict Flood

